

INDIAN INSTITUTE OF TECHNOLOGY DHARWAD

Department of Computer Science & Engineering Assignment 2

Assignment report submitted in fulfillment of the requirements for the degree of

Master of Technology in Computer Science and Engineering

Submitted by

GROUP NO. 16

Group Members: -

Name: ADITYA NARAYAN JENA
Roll: CS26MT004

Name: KIRAN CHAKRABARTI
Roll: CS26MT023

Name: SHIVAM PATEL Roll: CS26MT017

Under the supervision of

Prof. Dileep A. D.

Academic Year: 2026 - 2027

Abstract

This report presents an experimental study of fully connected neural networks (FCNNs) implemented from scratch in Python using backpropagation and pattern-mode stochastic gradient descent. The experiments cover three-class, two-dimensional classification on linearly separable (LS) and non-linearly separable (NLS) datasets, together with univariate and bivariate regression on the corresponding Assignment-I datasets. Classification uses one hidden layer for the linearly separable dataset and two hidden layers for the non-linearly separable dataset, with logistic and hyperbolic tangent (tanh) activations evaluated. Regression uses hyperbolic tangent hidden layers and a linear output neuron. The instantaneous loss function is squared error, the classification inputs are standardized using training-set statistics, and the best model is selected from the validation set. The final results show that both activations solve the linearly separable classification task perfectly, while the non-linearly separable task benefits strongly from the deeper FCNN: the best logistic model reaches 88.67% test accuracy and the best tanh model reaches 100%. For regression, the best univariate architecture, 1-5-1, obtains a test RMSE of 0.131105, while the best bivariate architecture, 2-8-5-1, obtains a test RMSE of 0.080508. The plots further show that hidden non-linear transformations allow the FCNN to represent the curved relationships that the single-neuron model from Assignment I could not capture.

Keywords— fully connected neural network, backpropagation, stochastic gradient descent, logistic activation, tanh activation, classification, regression, squared error.

Table Of Contents

I. INTRODUCTION **1**
II. EXPERIMENTAL SETUP **1**
A. Datasets and Data Partitioning 1
B. FCNN Architecture 3
C. Activation Functions and Loss 4
D. Backpropagation and Parameter Update 5
E. Weight Initialization and Preprocessing 5
F. Hyperparameters, Model Selection and Early stopping 6
III. CLASSIFICATION: EXPERIMENTAL STUDY AND INFERENCES **7**
A. Linearly Separable Dataset 7
B. Non-Linearly Separable Dataset 10
C. Classification Comparison with Assignment I 14
IV. REGRESSION: EXPERIMENTAL STUDY AND INFERENCES **14**
A. Univariate Regression 14
B. Bivariate Regression 18
C. Regression Comparison with Assignment I 22
V. NODE-OUTPUT ANALYSIS **22**
VI. DISCUSSION AND INFERENCES **23**
VII. CONCLUSION **23**
REFERENCES **23**

List of Figures

Figure	Caption
Fig. 1	Original linearly separable classification dataset in the two-dimensional feature space.
Fig. 2	Original non-linearly separable classification dataset in the two-dimensional feature space.
Fig. 3	Original univariate regression dataset.
Fig. 4	Original bivariate regression target surface.
Fig. 5	LS classification: logistic activation, average training and validation error versus epoch.
Fig. 6	LS classification: tanh activation, average training and validation error versus epoch.
Fig. 7	LS classification: selected logistic model decision region in the standardized X_1–X_2 feature space.
Fig. 8	LS classification: selected tanh model decision region in the standardized X_1–X_2 feature space.
Fig. 9	LS classification: test confusion matrix for the selected logistic model.
Fig. 10	LS classification: test confusion matrix for the selected tanh model.
Fig. 11	NLS classification: logistic activation, average training and validation error for the selected architecture.
Fig. 12	NLS classification: tanh activation, average training and validation error for the selected architecture.
Fig. 13	NLS classification: selected logistic model decision region in the X_1–X_2 feature space.
Fig. 14	NLS classification: selected tanh model decision region in the X_1–X_2 feature space.
Fig. 15	NLS classification: test confusion matrix for the selected logistic model.
Fig. 16	NLS classification: test confusion matrix for the selected tanh model.
Fig. 17	Univariate regression: average training error versus epoch for the selected 1-5-1 architecture.
Fig. 18	Univariate regression: training target and model output versus X(independent Variable).
Fig. 19	Univariate regression: validation target and model output versus X(independent Variable).
Fig. 20	Univariate regression: test target and model output versus X(independent Variable).
Fig. 21	Univariate regression: target output versus model output on training data.
Fig. 22	Univariate regression: target output versus model output on validation data.
Fig. 23	Univariate regression: target output versus model output on test data.
Fig. 24	Bivariate regression: average training error versus epoch for the selected 2-8-5-1 architecture.
Fig. 25	Bivariate regression: training target and model output surface.
Fig. 26	Bivariate regression: validation target and model output surface.
Fig. 27	Bivariate regression: test target and model output surface.
Fig. 28	Bivariate regression: target output versus model output on training data.
Fig. 29	Bivariate regression: target output versus model output on validation data.
Fig. 30	Bivariate regression: target output versus model output on test data.

List of Tables

Table	Caption
Table I	Dataset partitioning and sample distribution.
Table II	FCNN architecture families and activation functions used for the experiments.
Table III	Experimental hyperparameters, preprocessing, initialization, and training configuration.
Table IV	Linearly Separable classification: Logistic activation — validation architecture comparison
Table V	Linearly Separable classification: Tanh activation — validation architecture comparison
Table VI	Non-Linearly Separable classification: Logistic activation — validation architecture comparison
Table VII	Non-Linearly Separable classification: Tanh activation — validation architecture comparison
Table VIII	Comparison of classification performance between Assignment I and Assignment II.
Table IX	Univariate regression architecture comparison using RMSE and percentage RMSE.
Table X	Bivariate regression architecture comparison using RMSE and percentage RMSE.

I. INTRODUCTION

The objective of this assignment is to extend the concepts of the single-neuron learning model studied in Assignment I to a fully connected neural network capable of learning nonlinear representations. The implementation was developed without any neural-network training library. A reusable implementation consisting of dense layers, activation layers, a squared-error loss, an FCNN model container, and an SGD trainer is used for all experiments.

Four experimental cases are considered. The first two experiments are on three-class, two-dimensional classification problems: a linearly separable dataset and a non-linearly separable dataset. The linearly separable classifier contains one hidden layer, whereas the non linearly separable classifier contains two hidden layers. For both datasets, the hidden-node counts are varied and both logistic and tanh activations are evaluated. The remaining two cases are regression problems: a one-input univariate dataset and a two-input bivariate dataset. The regression models use tanh in the hidden layer or hidden layers and a linear output unit.

The central questions for this experiment are how hidden-layer capacity affects representation, whether the activation function changes convergence and predictive performance, and how the FCNN compares with the single-neuron model used previously. The report therefore combines numerical metrics with feature-space decision regions, error curves, target-versus-model plots, regression surfaces, and node-output visualizations.

II. EXPERIMENTAL SETUP

A. Datasets and Data Partitioning

The two classification datasets contain three classes and two input features, with 500 samples in each class. Thus, each classification dataset contains 1500 observations. The samples are partitioned independently within each class using a 60:20:20 training-validation-test split, giving 300 training, 100 validation, and 100 test samples per class, or 900, 300, and 300 samples overall respectively. The regression datasets contain 1001 sample points for the univariate problem and 10201 sample points for the bivariate problem. The same 60:20:20 split gives 601, 200, 200 samples for the univariate data and 6121/2040/2040 samples for the bivariate data respectively.

Table I: Data Partition

Dataset	Inputs	Samples	Training	Validation	Test
Linearly separable classification	2	1500	900	300	300
Non-linearly separable classification	2	1500	900	300	300
Univariate regression	1	1001	601	200	200
Bivariate regression	2	10201	6121	2040	2040

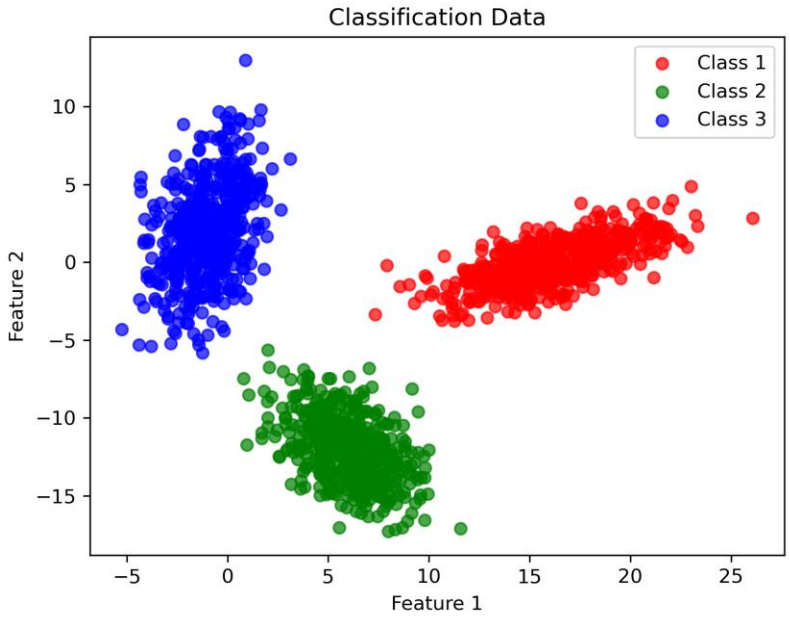


Fig. 1. Original linearly separable classification dataset.

The linearly separable feature plot shows three compact and well-separated clusters. The separation visible directly in feature space indicates that relatively simple decision boundaries should be sufficient.

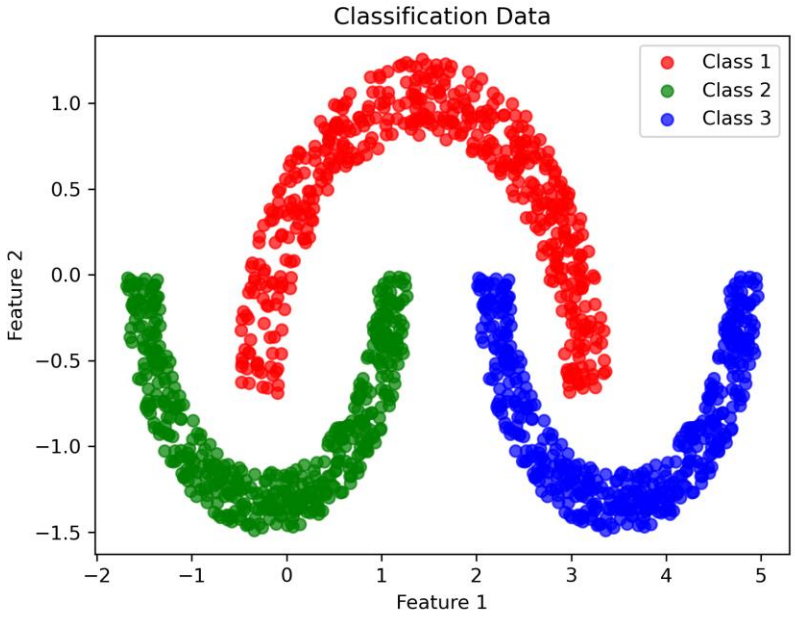


Fig. 2. Original non-linearly separable classification dataset.

The non-linearly separable feature plot contains U-shaped, interleaved class distributions. The geometry demonstrates why a linear classifier is insufficient and motivates the use of hidden layers for nonlinear representation.

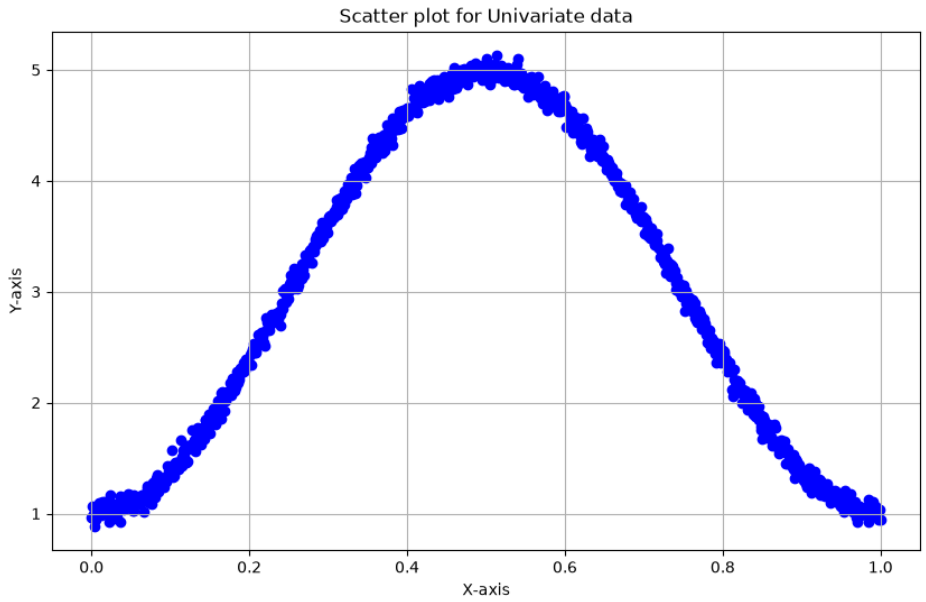


Fig. 3. Original univariate regression dataset.

The univariate target follows a smooth single-hump nonlinear relationship with the input X. A model with only a linear transformation or linear activation function cannot reproduce this shape.

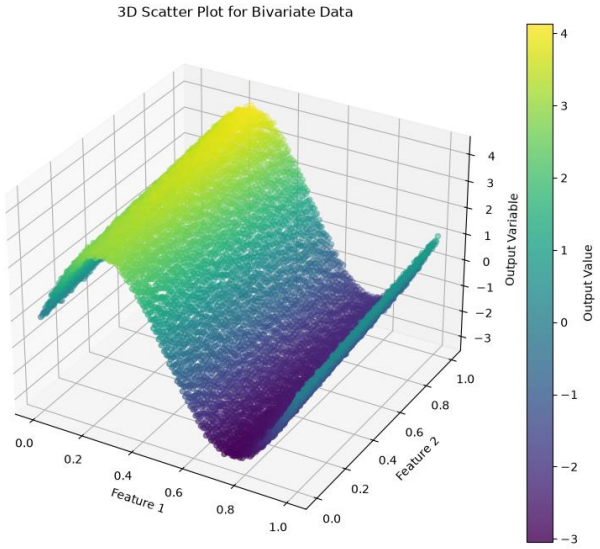


Fig. 4. Original bivariate regression target surface.

The bivariate target varies nonlinearly over X1 and X2. The curved surface indicates the requirement of non-linear hidden transformations for accurate approximation.

B. FCNN Architecture

For an input vector, a bias component is appended before a dense transformation. Let $\tilde{\mathbf{x}}^{(l-1)}$ denote the augmented input to layer l, and let $\mathbf{W}^{(l)}$ denote the corresponding weight matrix. The forward pass is written as

$$\mathbf{a}^{(l)} = (\mathbf{W}^{(l)})^T \mathbf{x}^{(l-1)}$$

$$y^{(l)} = f^{(l)}(\mathbf{a}^{(l)})$$

Here, **x**, **z** and **a** are vectors and **W** is a matrix. The network output is denoted by ŷ. For classification, the output vector is converted to a class label using the index of its largest component. The linearly separable architectures are 2–H–3 with H ∈ {3, 5, 8, 10}; the non-linearly separable architectures are 2–H1–H2–3 with (H1,H2) ∈ {(3,3), (5,5), (8,5), (10,8)}. Regression uses 1–H–1 for the univariate dataset where H ∈ {3, 5, 8, 10} and 2-H-1 with H ∈ {3, 5, 8} or 2-H1-H2-1 with (H1,H2) ∈ {(5,5), (8,5)} for the bivariate dataset.

Table II: FCNN architecture families and activation functions used for the experiments.

Task	Architecture family	Activation
linearly separable classification	2-H-3	Logistic / tanh
non-linearly separable classification	2-H1-H2-3	Logistic / tanh
Univariate regression	1-H-1	Tanh and Linear
Bivariate regression	2-H-1; 2-H1-H2-1	Tanh and Linear

C. Activation Functions and Loss

The logistic sigmoid and hyperbolic tangent (tanh) activation functions are used in the classification networks. Their mathematical forms and derivatives are given by

Logistic sigmoid:

$$\sigma(a) = 1 / (1 + e^{-a}) \qquad \sigma'(a) = \sigma(a)[1 - \sigma(a)]$$

Hyperbolic tangent:

$$\tanh(a) = (e^{a} - e^{-a}) / (e^{a} + e^{-a}) \qquad \tanh'(a) = 1 - \tanh^2(a)$$

For regression, the hidden layers use the tanh activation, while the output neuron uses a linear activation:

$$f(a) = a \quad f'(a) = 1$$

For training pattern n, the instantaneous squared-error loss used in the experiments is

$$E_n = \tfrac{1}{2} \Sigma_{k=1}^{K} (y_{nk} - \hat{y}_{nk})^2$$

where y_{nk} denotes the desired output for output neuron k, and $\hat{y}_{nk}$ denotes the corresponding network output. The factor ½ is included to simplify differentiation.

Differentiating the loss with respect to the network output gives

$$\partial E_n / \partial \hat{y}_{nk} = -(y_{nk} - \hat{y}_{nk})$$

Thus, the derivative can equivalently be written as $\hat{y}_{nk} - y_{nk}$. The subsequent error-signal formulation follows the sign convention used in the FCNN derivation provided for this assignment.

D. Backpropagation and Parameter Update

For a training pattern n, let $a_{nk}^{(L)}$ denote the pre-activation of output neuron k in the final layer L, and let $\hat{y}_{nk}$ denote its output. Following the error-signal convention used in the FCNN derivation, the output-layer error signal is defined as

$$\delta_{nk}^{(L)} = -(y_{nk} - \hat{y}_{nk})\, f'(a_{nk}^{(L)})$$

where f′ represents the derivative of the activation function used by the output neuron.

For a hidden neuron j in layer l, the error signal is propagated from the following layer as

$$\delta_{nj}^{(l)} = f'(a_{nj}^{(l)}) \sum_{k=1}^{K} w_{jk}^{(l+1)}\, \delta_{nk}^{(l+1)}$$

Thus, the error signal at a hidden neuron depends on the error signals of the neurons in the next layer and the corresponding connecting weights.

For a weight connecting neuron j in the preceding layer to neuron k in layer l, the instantaneous weight change is

$$\Delta w_{kj}^{(l)} = -\eta\, \delta_{nk}^{(l)}\, h_{nj}^{(l-1)}$$

where $h_{nj}^{(l-1)}$ is the output of neuron j in the preceding layer and η is the learning rate.

For the complete weight matrix, the update can be expressed as

$$\Delta W^{(l)} = -\eta\, \delta_{n}^{(l)}\, (h_{n}^{(l-1)})^{T}$$

where $W^{(l)}$ is the weight matrix and $S_n^{(l)}$ and $h_n^{(l-1)}$ are the corresponding error-signal and activation vectors.

The weights are then updated according to

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} + \Delta\mathbf{W}^{(l)}$$

or equivalently,

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta\, \delta_{n}^{(l)}\, (h_{n}^{(l-1)})^{T}$$

This update is performed immediately after each training pattern, implementing pattern-mode stochastic gradient descent.

At the beginning of every epoch, the training patterns are randomly shuffled. Each pattern is then forwarded through the network, its instantaneous squared error is calculated, and the error is propagated backward through the network. The weights are updated immediately before processing the next pattern. After all training patterns have been processed, the average training error is calculated and recorded for that epoch.

This formulation follows the derivation supplied for the assignment, where the output error signal is formed using the difference between the desired output and network output, followed by propagation through the activation derivatives and connecting weights.

E. Weight Initialization and Preprocessing

The weights of each dense layer are initialized using a zero-mean normal distribution whose scale depends on the number of inputs to the layer. For a dense layer with d input connections, the initialization is

$$w_{ij} \sim N(0,1) / \sqrt{d}$$

where d is the input dimension of the corresponding dense layer. The dense layer also contains an additional bias weight.

The scaling by $1/\sqrt{d}$ is used to control the magnitude of the weighted sum entering each neuron. Without scaling, increasing the number of input connections can increase the variance of the pre-activation and produce excessively large values. For sigmoid and tanh activations, very large positive or negative pre-activation values can move neurons into their saturation regions, where the derivatives become very small and learning can become slow.

The chosen initialization also avoids the symmetry problem associated with initializing all weights to zero. If all neurons in a hidden layer started with identical zero weights, they would receive identical gradients and consequently learn identical representations.

Therefore, the $1/\sqrt{d}$ scaling provides a simple variance-control mechanism that keeps the magnitude of the initial activations reasonably stable as the number of incoming connections changes. This is particularly useful in the present experiments because multiple architectures with different hidden-node counts are evaluated using sigmoid and tanh nonlinearities.

For classification, the stratified train-validation-test split is performed first. The mean and standard deviation required for standardization are calculated only from the training set. The same training-set statistics are then applied to the validation and test sets. This prevents information from the validation and test sets from entering the preprocessing stage.

The standardized feature for an input variable x is calculated as

$$x' = (x - \mu_{train}) / \sigma_{train}$$

where μ_{train} and σ_{train} are the mean and standard deviation calculated from the training set.

The regression implementation retains the input variables in their original dataset ranges.

F. Hyperparameters, Model Selection and Early stopping

Table III: Experimental hyperparameters, preprocessing, initialization, and training configuration.

Parameter	Classification	Regression
Learning rate	0.01	0.001
Maximum epochs	1000	1000
Instantaneous loss used	Squared error	Squared error
Data Split	60/20/20	60/20/20
Input preprocessing	Training-set standardization	Original input range
Training mode	Pattern-mode SGD + epoch shuffle	Pattern-mode SGD + epoch shuffle
Selection criterion	Lowest validation error	Lowest validation RMSE
Initialization	Scaled normal, $1/\sqrt{d}$	Scaled normal, $1/\sqrt{d}$

At the beginning of every epoch, the training indices are randomly shuffled. The network then presents each training pattern once and updates the weights immediately after that pattern. This implements the pattern-mode stochastic gradient descent. Validation error is computed at the end of each epoch, and the parameter state giving the lowest validation error is stored and restored after training.

Early stopping criterions are the maximum number of epoch count, the average-error threshold, the successive-epoch convergence threshold i.e. absolute difference successive epochs is reached, or validation-based early stopping when validation error increases. Consequently, the reported best architecture is selected based on validation performance rather than test performance.

III. CLASSIFICATION: EXPERIMENTAL STUDY AND INFERENCES

A. Linearly Separable Dataset

The linearly separable dataset is structurally simple: its three classes form compact, well-separated clusters. All four architectures achieve 100% validation accuracy for both activations. The selected models are 2-5-3 for logistic activation and 2-10-3 for tanh activation, based on the lowest validation error within each activation experiment.

Table IV: Linearly Separable classification: Logistic activation — validation architecture comparison

Architecture	Accuracy	Mean P	Mean R	Mean F1	Best epoch	Best val. error
2-3-3	100.00%	1.0000	1.0000	1.0000	635	0.000501
2-5-3	100.00%	1.0000	1.0000	1.0000	642	0.000465
2-8-3	100.00%	1.0000	1.0000	1.0000	604	0.000569
2-10-3	100.00%	1.0000	1.0000	1.0000	601	0.000478

Table V: Linearly Separable classification: Tanh activation — validation architecture comparison

Architecture	Accuracy	Mean P	Mean R	Mean F1	Best epoch	Best val. error
2-3-3	100.00%	1.0000	1.0000	1.0000	254	0.000191
2-5-3	100.00%	1.0000	1.0000	1.0000	242	0.000163
2-8-3	100.00%	1.0000	1.0000	1.0000	41	0.001065
2-10-3	100.00%	1.0000	1.0000	1.0000	244	0.000133

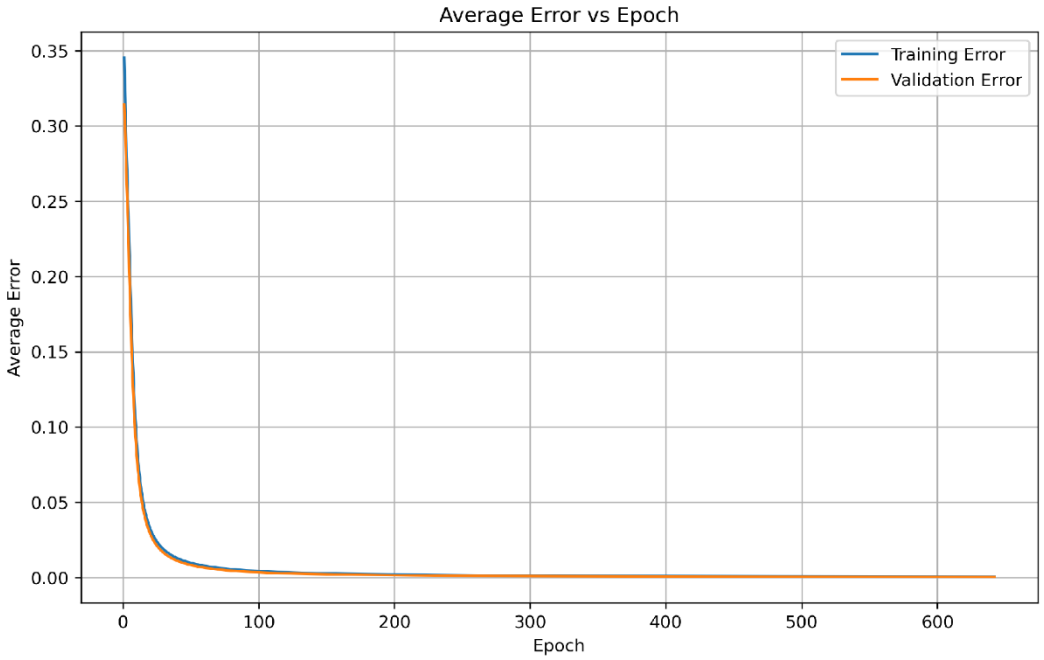


Fig. 5. Linearly separable classification for logistic activation of best architecture: Average training and validation error versus Epoch.

The logistic error curve decreases rapidly at the beginning and then approaches zero as it can be observed in Fig. 5. The long convergence period indicates that the model can solve the linearly separable geometry but requires many stochastic updates before reaching the stopping condition. The validation curve follows the training curve closely, showing no visible generalization gap.

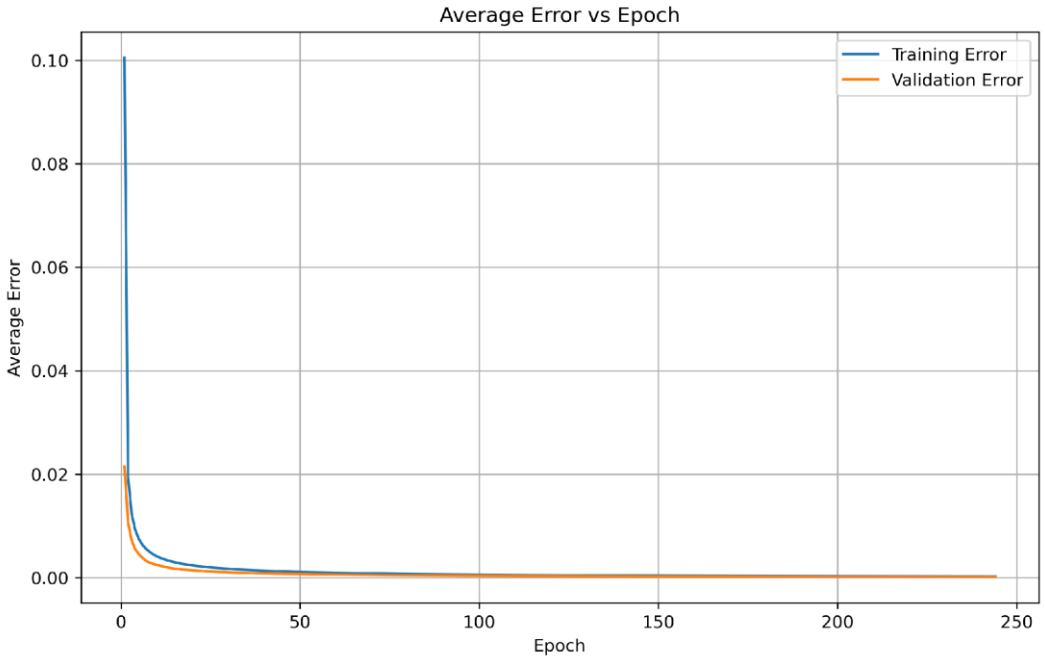


Fig. 6. Linearly separable classification for tanh activation of best architecture: Average training and validation error versus Epoch.

The tanh curve also falls to a very small error, but it reaches its best validation states substantially earlier than the logistic run as depicted in Fig. 6. This indicates faster optimization for the standardized linearly separable features, even though both activations ultimately obtain the same classification accuracy.

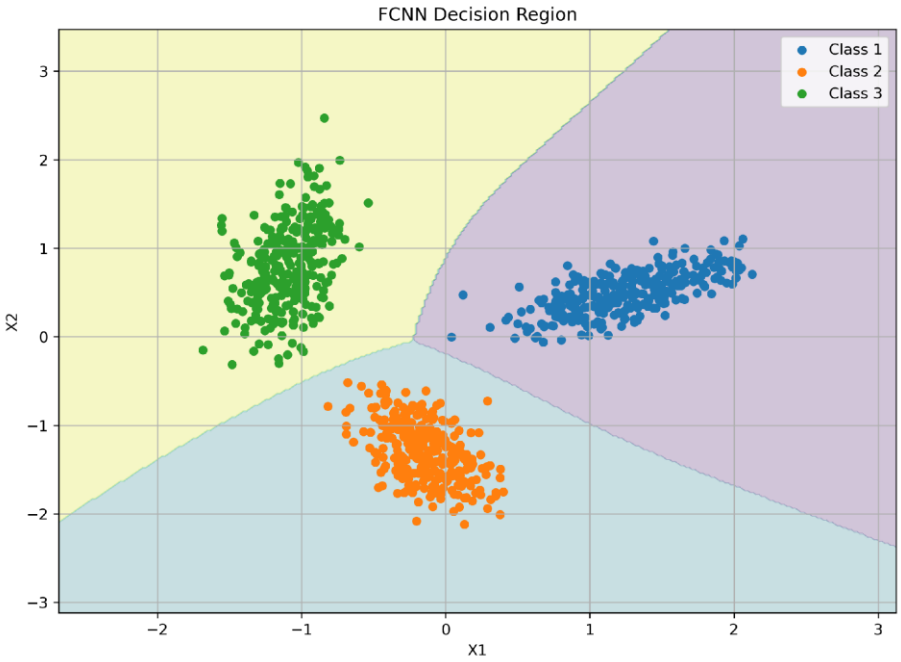


Fig. 7. Linearly separable classification: selected logistic model decision region in the standardized X1-X2 feature space.

The feature-space decision region forms three clean regions around the three clusters as displayed in Fig. 7. The boundaries remain simple because the data themselves are linearly separable. The plot therefore confirms that the FCNN is not required to construct a complicated non-linear boundary for this dataset.

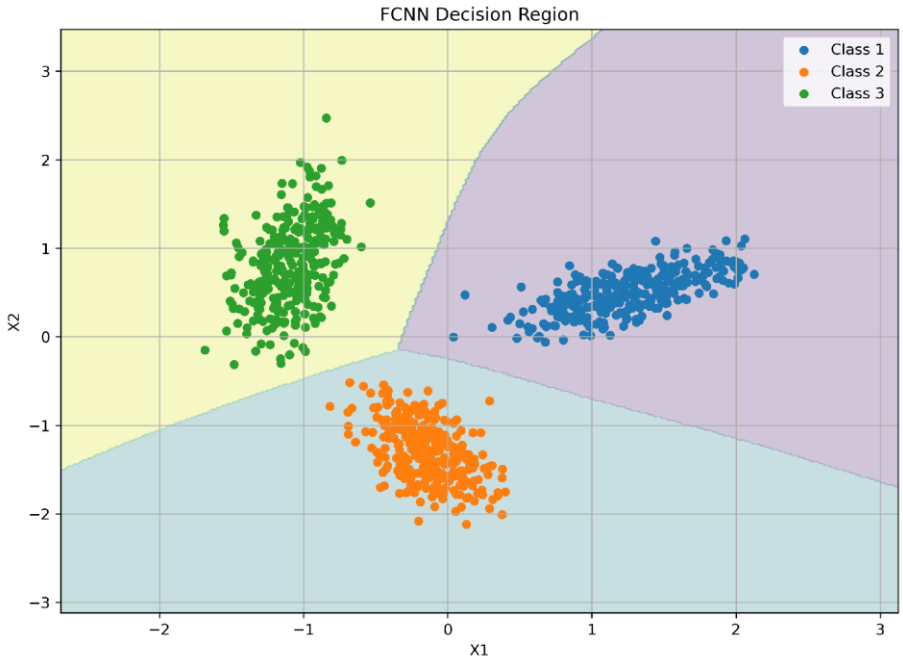


Fig. 8. Linearly separable classification: selected tanh model decision region in the standardized X1-X2 feature space.

The tanh model produces an equally clean feature-space partition as in Fig. 8. The agreement between the logistic and tanh regions and their identical 100% test accuracy shows that activation choice has little effect on the final classification outcome when the classes are already well separated.

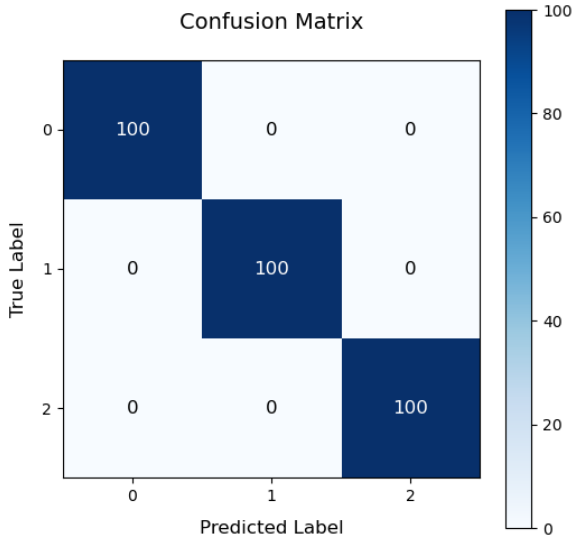


Fig. 9. Linearly separable classification: test confusion matrix for the selected logistic model.

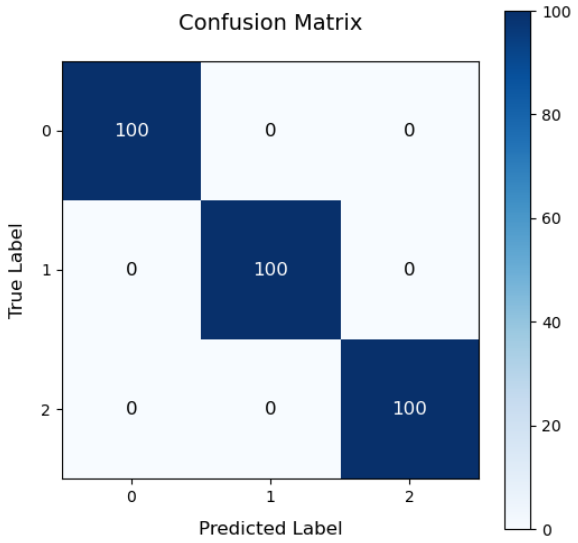


Fig. 10. Linearly separable classification: test confusion matrix for the selected tanh model.

Both test confusion matrices are diagonal with 100 correct predictions for every class. Hence accuracy, per-class precision, per-class recall, and per-class F1-score are all 1.00 for both selected models.

B. Non-Linearly Separable Dataset

The non-linearly separable dataset contains interleaved U-shaped curved classes, so the classifier must learn a non-linear feature representation before forming the final class regions. The two-hidden-layer architectures were therefore evaluated with both activation functions.

Table VI: Non-Linearly Separable classification: Logistic activation — validation architecture comparison

Architecture	Accuracy	Mean P	Mean R	Mean F1	Best epoch	Best val. error
2-3-3-3	88.67%	0.8861	0.8867	0.8847	303	0.089481
2-5-5-3	87.67%	0.8758	0.8767	0.8743	116	0.094874
2-8-5-3	89.67%	0.9005	0.8967	0.8932	110	0.087771
2-10-8-3	88.33%	0.8827	0.8833	0.8811	95	0.092309

Table VII: Non-Linearly Separable classification: Tanh activation — validation architecture comparison

Architecture	Accuracy	Mean P	Mean R	Mean F1	Best epoch	Best val. error
2-3-3-3	91.00%	0.9185	0.9100	0.9062	6	0.325353
2-5-5-3	88.33%	0.8829	0.8833	0.8812	2	0.367305
2-8-5-3	89.67%	0.9005	0.8967	0.8932	2	0.350432
2-10-8-3	100.00%	1.0000	1.0000	1.0000	33	0.003760

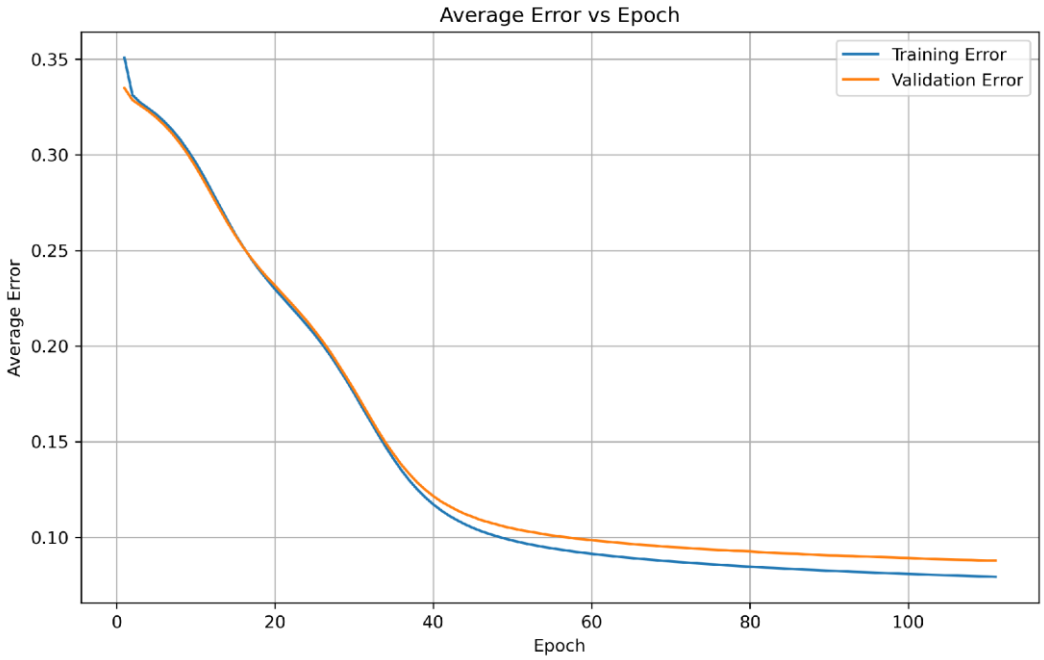


Fig. 11. Non-linearly separable classification for logistic activation of best architecture: Average training and validation error vs epochs.

The selected logistic architecture, 2-8-5-3, reduces the error substantially but converges much more slowly than the selected tanh model. The best validation state occurs at epoch 110. The longer training trajectory reflects the difficulty of representing the curved class boundaries with the logistic activation under the given optimization settings.

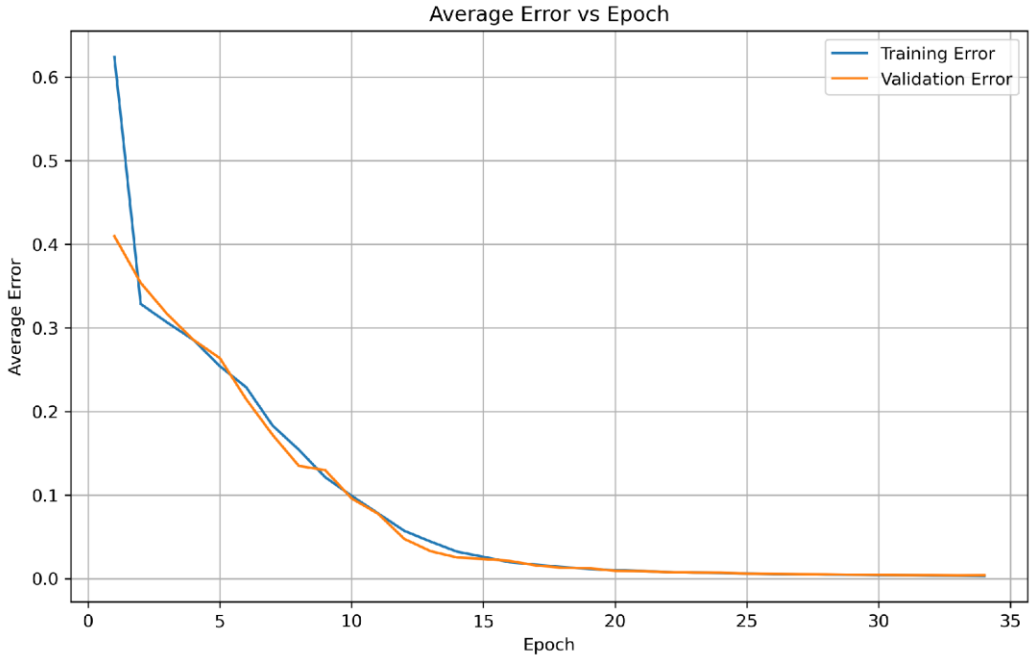


Fig. 12. Non-linearly separable classification for tanh activation of best architecture: Average training and validation error vs Epochs.

The selected tanh architecture, 2-10-8-3, reaches a validation error of 0.003760 at epoch 33. The sharp reduction in error and early best epoch show a much faster and more successful optimization than the logistic counterpart.

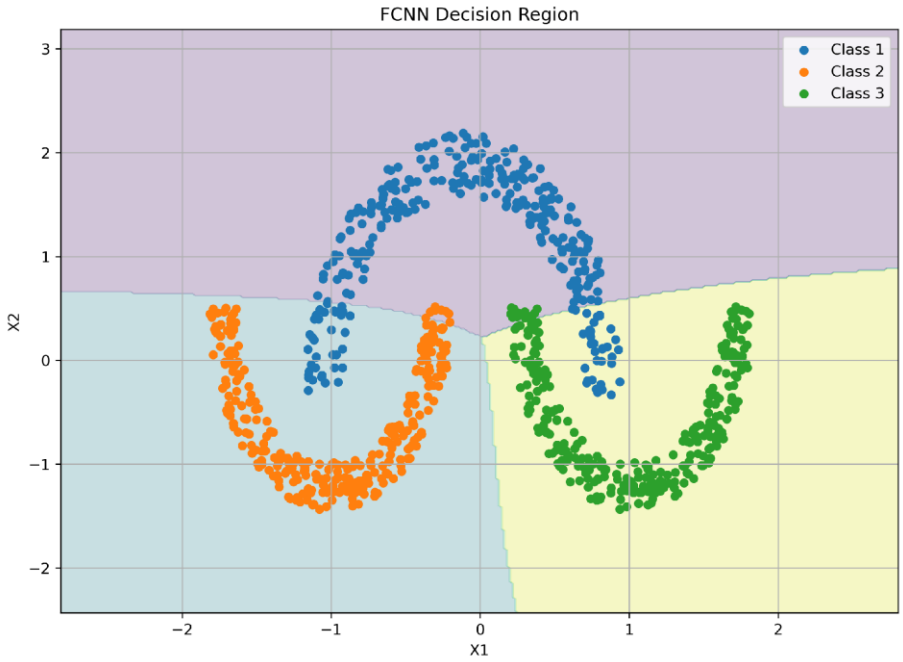


Fig. 13. Non-linearly separable classification: selected logistic model decision region in the X1-X2 feature space.

The logistic model produces curved decision regions rather than the straight boundaries obtained from the single-neuron Assignment-I model. Nevertheless, the class-1 region, as can be observed in Fig. 13, is not completely isolated from the other two classes, which is reflected in the residual classification errors.

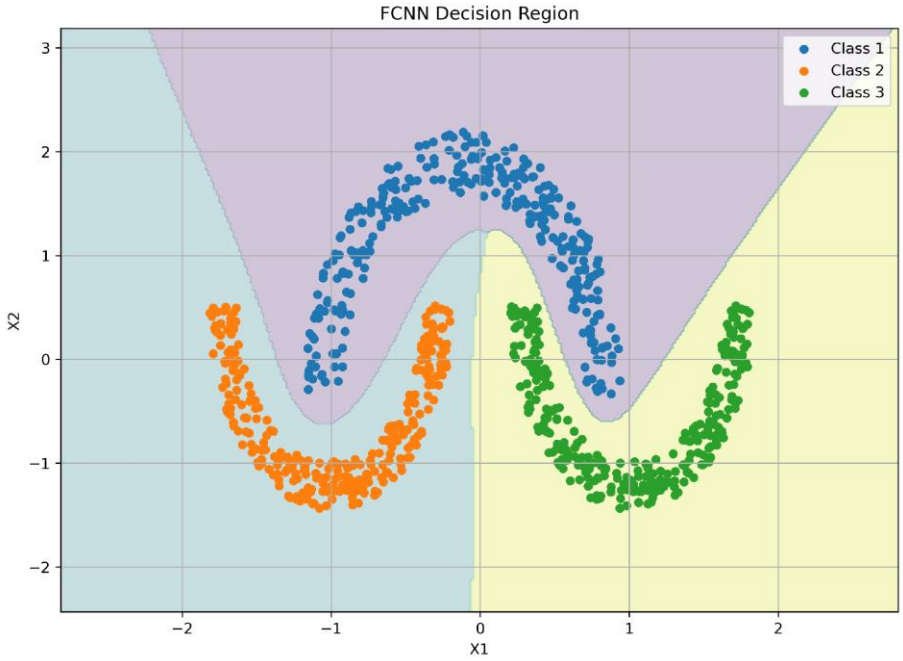


Fig. 14. Non-linearly separable classification: selected tanh model decision region in the X1-X2 feature space.

The tanh model closely follows the two-dimensional curved geometry. The three regions align with the interleaved class distributions, explaining the perfect test confusion matrix as in Fig. 14.

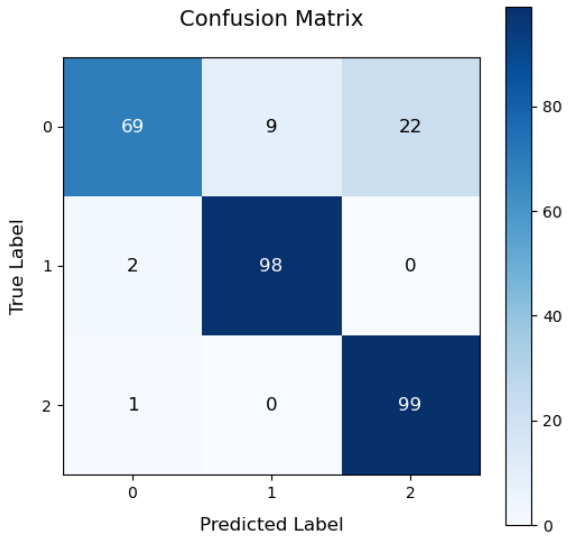


Fig. 15. Non-linearly separable classification: test confusion matrix for the selected logistic model.

The logistic test confusion matrix contains 69 correct Class-1 (represented as class-0 in confusion matrix) predictions, 98 correct Class-2 (represented as class-1 in confusion matrix) predictions, and 99 correct Class-3 (represented as class-2 in confusion matrix) predictions. Class 1 is the main source of error, with 9 samples assigned to Class 2 and 22 to Class 3. The resulting accuracy is 88.67%, with macro precision 0.8975, macro recall 0.8867, and macro F1-score 0.8817.

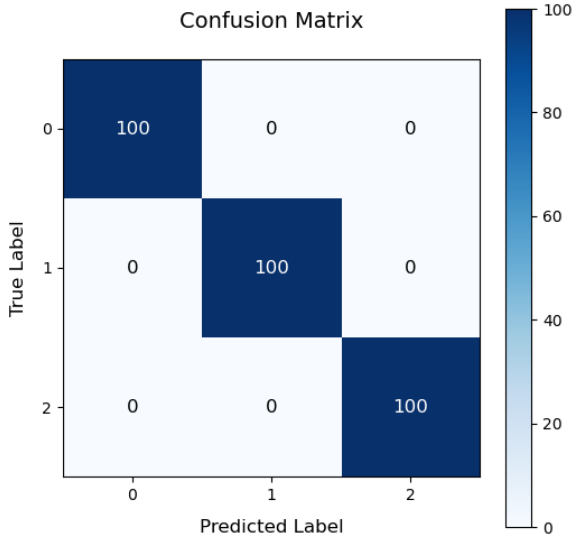


Fig. 16. Non-linearly separable classification: test confusion matrix for the selected tanh model.

The tanh test confusion matrix is perfectly diagonal, giving 100 correct predictions for each class. Accuracy, macro precision, macro recall, and macro F1-score are all 1.00. The improvement over the logistic non-linearly separable model demonstrates the capability of both non-linear depth and the tanh activation for this particular dataset.

C. Classification Comparison with Assignment I

Table VIII: Comparison of classification performance between Assignment I and Assignment II.

Dataset / model	Assignment I test accuracy	Assignment II test accuracy	Change
Linearly Separable logistic	100.00%	100.00%	0.00 percentage points
Linearly Separable tanh	100.00%	100.00%	0.00 percentage points
Non-Linearly Separable logistic	86.22%	88.67%	+2.45 percentage points
Non-Linearly Separable tanh	88.00%	100.00%	+12.00 percentage points

The linearly separable results remain perfect because the original feature geometry is already linearly separable. The most important change occurs for non-linearly separable classification. Assignment I used a single-neuron/One-against-One decision mechanism, which could not represent the curved boundary adequately. Assignment II introduces hidden nonlinear transformations and directly learns a three-output FCNN, allowing the tanh model to recover the curved class structure and increase test accuracy from 88.00% to 100.00%. The logistic model also improves, but its remaining Class-1 confusion shows that the activation and optimization configuration still affects how effectively the available hidden representation is used.

IV. REGRESSION: EXPERIMENTAL STUDY AND INFERENCES

A. Univariate Regression

The univariate dataset contains one input X and one continuous target. Four one-hidden-layer architectures were evaluated. The results show a clear capacity effect: the 1-3-1, 1-8-1, and 1-10-1 runs terminate after comparatively little improvement, whereas the 1-5-1 architecture continues to a much lower validation error and is selected as the best model.

Table IX: Univariate regression architecture comparison using RMSE and percentage RMSE.

Architecture	Train RMSE	Train %RMSE	Validation RMSE	Validation %RMSE	Best epoch
1-3-1	1.397176	46.566%	1.335250	45.400%	3
1-5-1	0.129626	4.320%	0.128448	4.367%	85
1-8-1	1.398271	46.602%	1.343442	45.678%	2
1-10-1	1.373420	45.774%	1.316801	44.773%	10

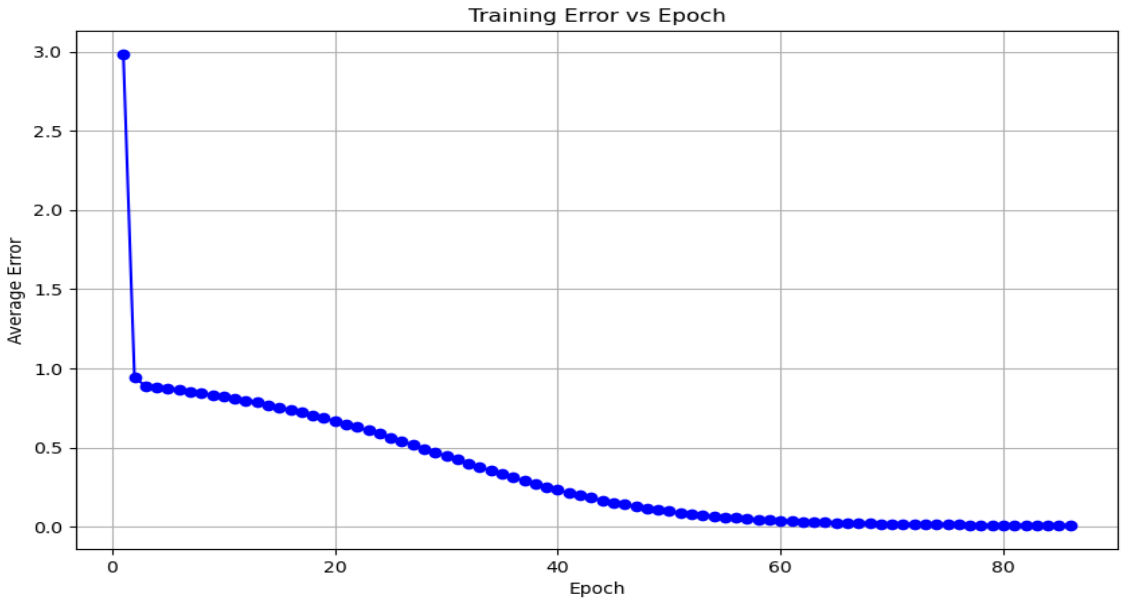


Fig. 17. Univariate regression: Average training error versus Epoch for the selected 1-5-1 architecture.

The selected model's training error decreases substantially across the run and reaches a low-error region rather than becoming trapped at the high plateau as observed in the weaker architectures. The best validation state occurs at epoch 85, indicating that continued optimization was useful for this architecture.

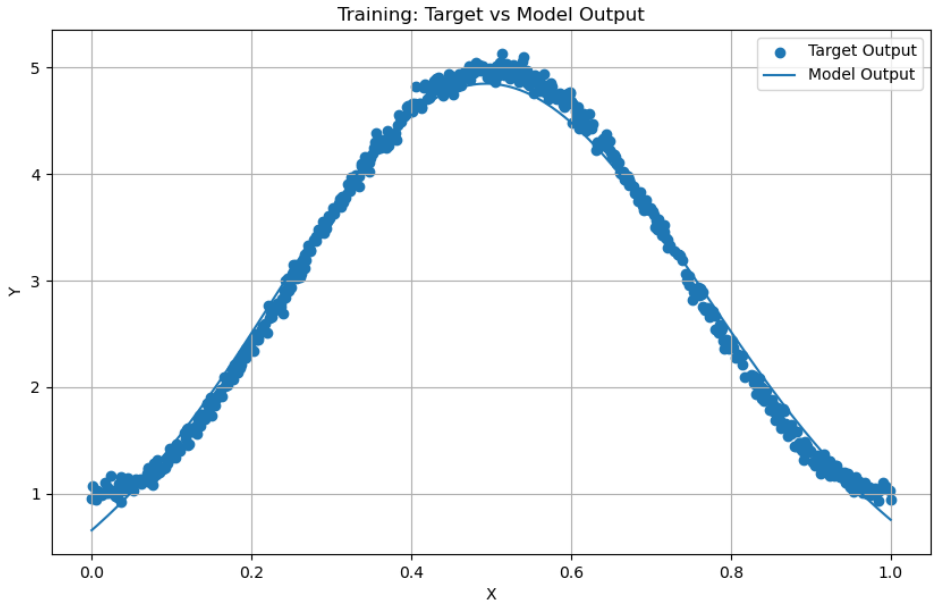


Fig. 18. Univariate regression: training target output and model output versus X(independent Variable).

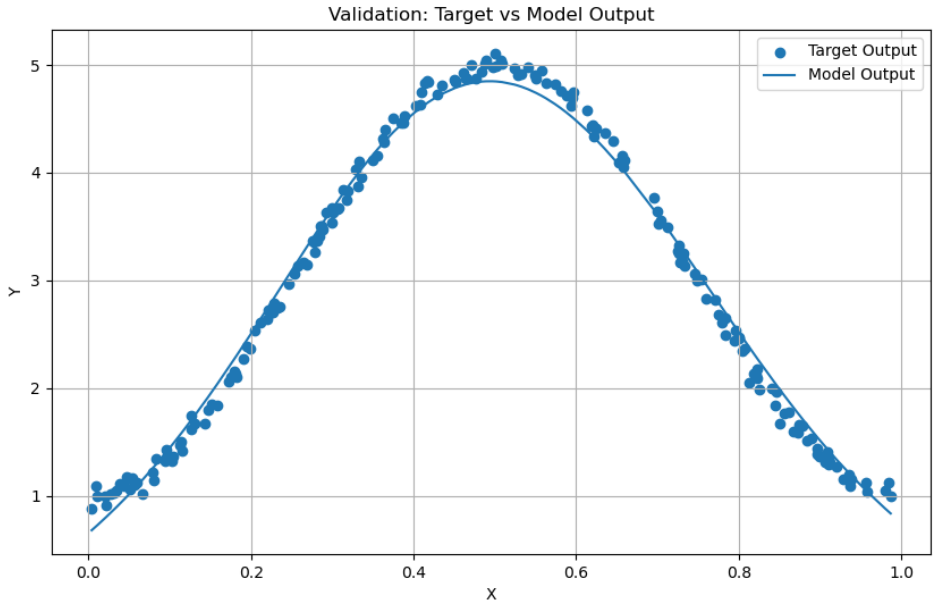


Fig. 19. Univariate regression: validation target output and model output versus X(independent Variable).

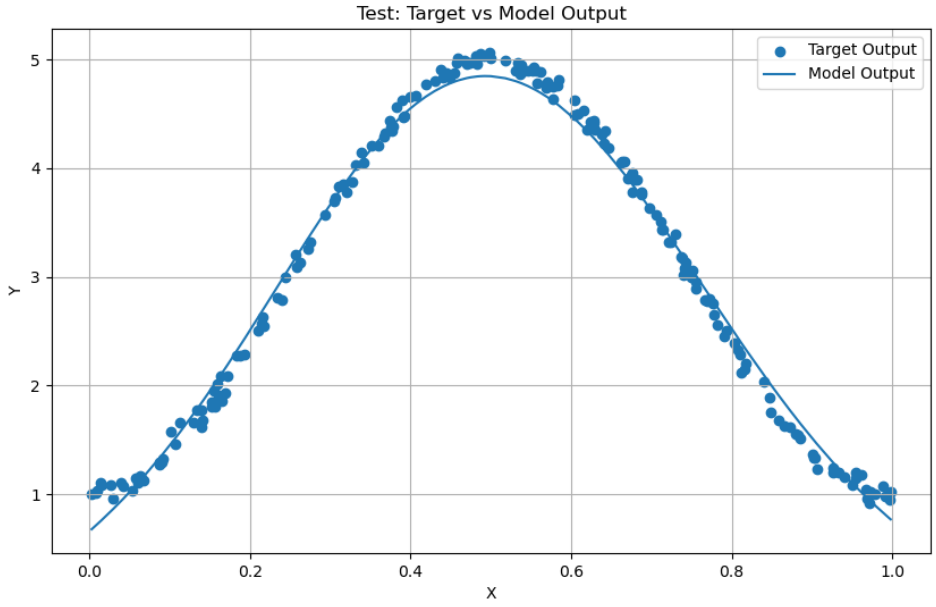


Fig. 20. Univariate regression: test target output and model output versus X(independent Variable).

Across training, validation, and test data, the predicted curve follows the smooth target hump closely. The model therefore learns the non-linear dependence rather than collapsing to a constant prediction. The small difference between the validation RMSE (0.128448) and test RMSE (0.131105), as observed in the Architecture Comparison Table, indicates model generalization.



Fig. 21. Univariate regression: target output versus model output on training data.

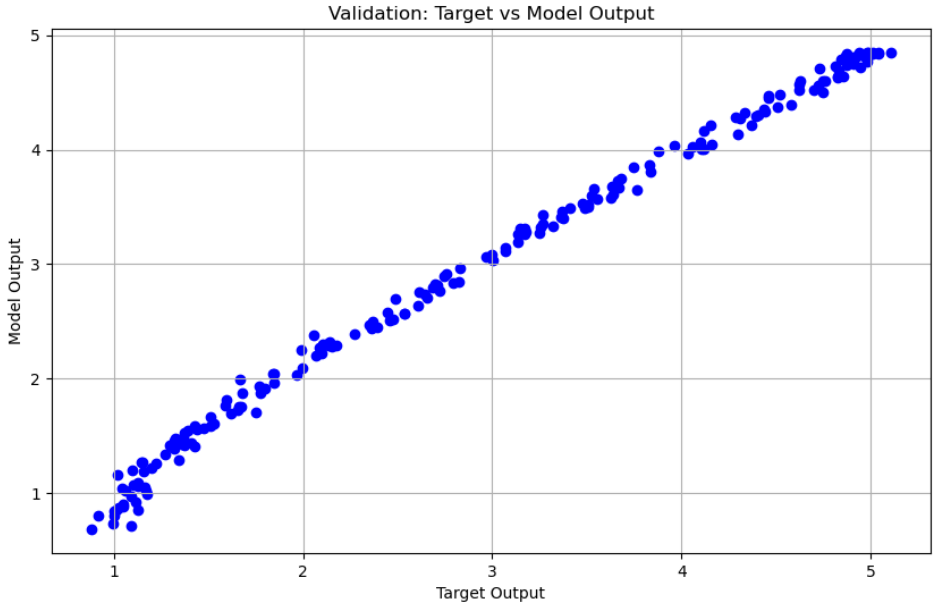


Fig. 22. Univariate regression: target output versus model output on validation data.

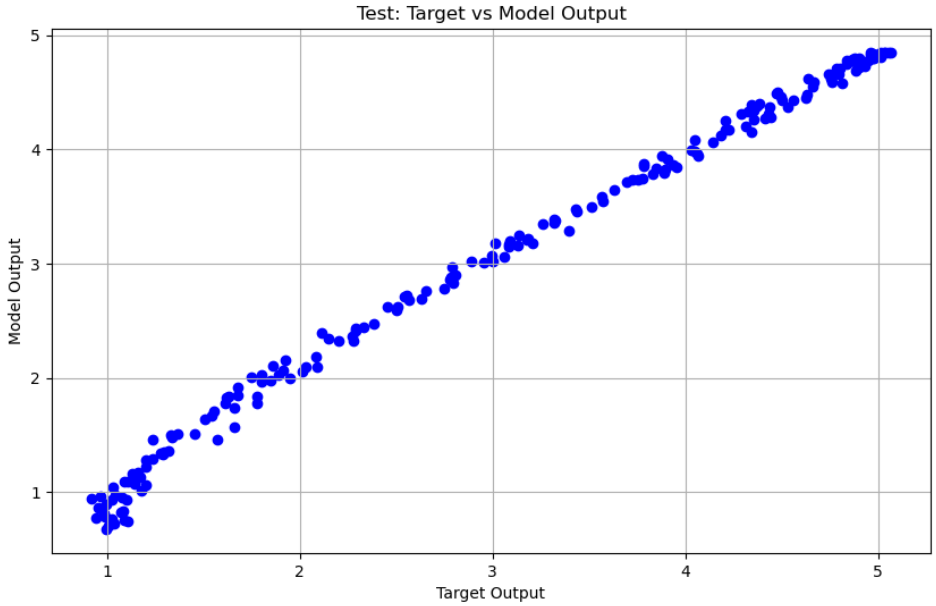


Fig. 23. Univariate regression: target output versus model output on test data.

The three target-versus-model scatter plots lie close to the diagonal relationship expected from accurate regression. The consistency across all three partitions supports the numerical RMSE results and indicates that the selected model is not merely fitting the training set.

B. Bivariate Regression

The bivariate regression problem contains two inputs, X1 and X2, and a continuous target. Three one-hidden-layer and two two-hidden-layer architectures were evaluated. The two-hidden-layer models substantially outperform the one-hidden-layer alternatives, demonstrating that higher depth is useful for approximating the non-linear target surface.

Table X: Bivariate regression architecture comparison using RMSE and percentage RMSE.

Architecture	Train RMSE	Train %RMSE	Validation RMSE	Validation %RMSE	Best epoch
2-3-1	0.491268	97.745%	0.485128	89.255%	30
2-5-1	0.406362	80.851%	0.401636	73.894%	26
2-8-1	0.415429	82.655%	0.409356	75.314%	25
2-5-5-1	0.082675	16.449%	0.080279	14.770%	30
2-8-5-1	0.081752	16.266%	0.079254	14.581%	42

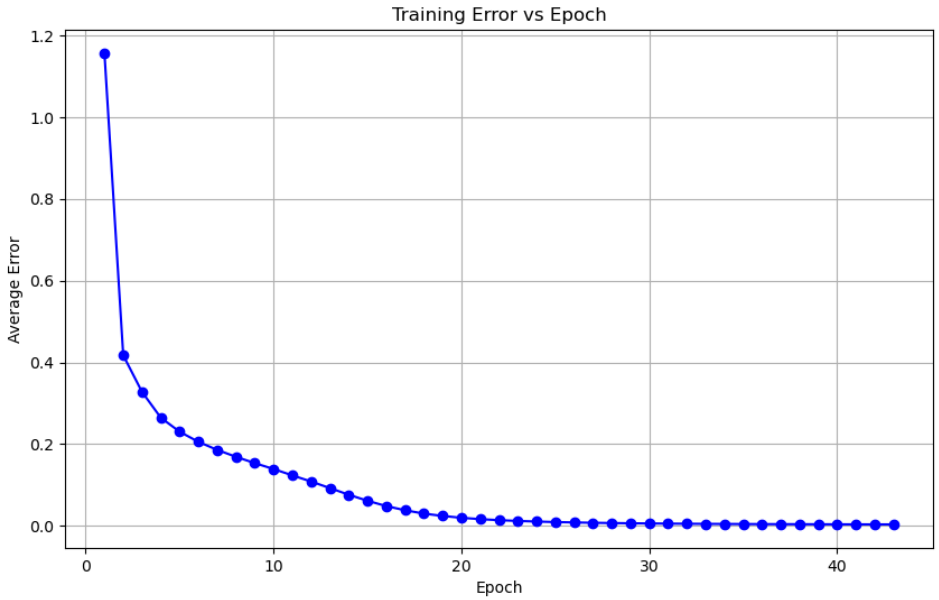


Fig. 24. Bivariate regression: Average training error versus Epoch for the selected 2-8-5-1 architecture.

The selected 2-8-5-1 model reduces the training error rapidly and continues improving until its best validation state at epoch 43. The deeper architectures reach an error level far below the one-hidden-layer models, indicating that the performance gain is primarily associated with representation capacity.

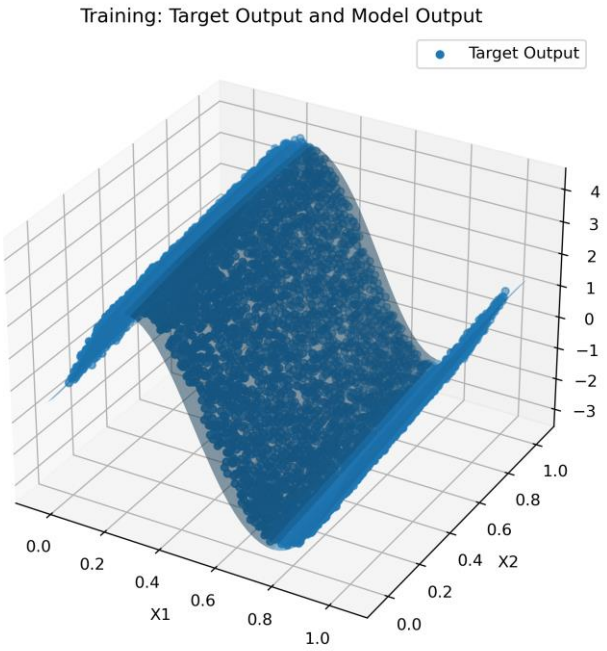


Fig. 25. Bivariate regression: training target and model output surface.

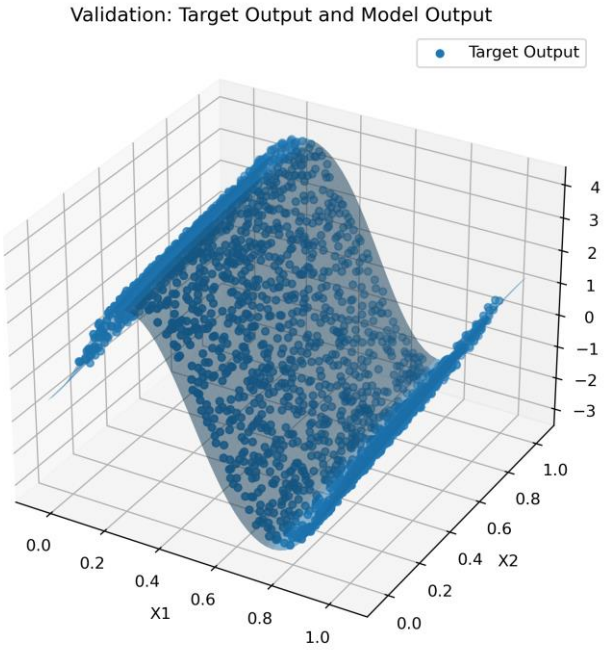


Fig. 26. Bivariate regression: validation target and model output surface.

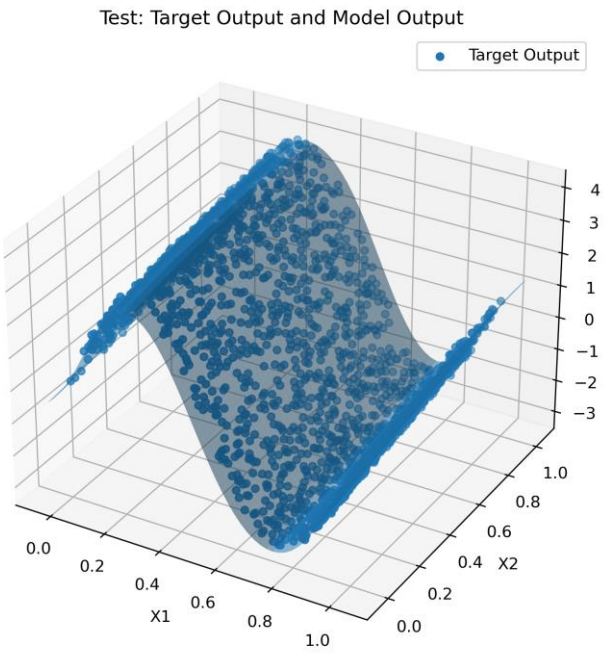


Fig. 27. Bivariate regression: test target and model output surface.

The three-dimensional plots (Fig. 25, Fig. 26, Fig. 27) show that the predicted surface follows the major curvature of the target surface over X1 and X2. The close visual agreement between training, validation, and test surfaces is consistent with the low RMSE values and indicates that the non-linear relationship has been learned rather than memorized.



Fig. 28. Bivariate regression: target output versus model output on training data.

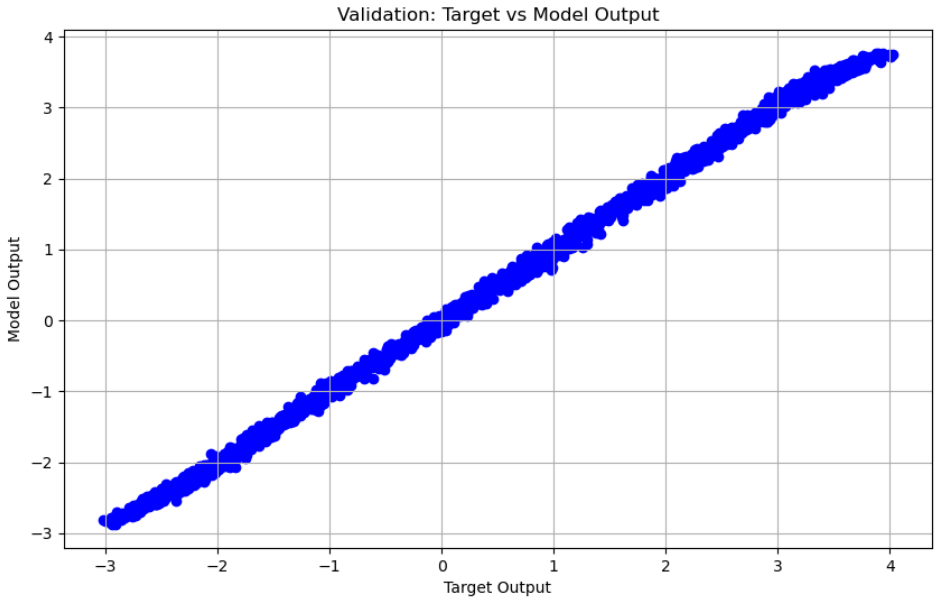


Fig. 29. Bivariate regression: target output versus model output on validation data.

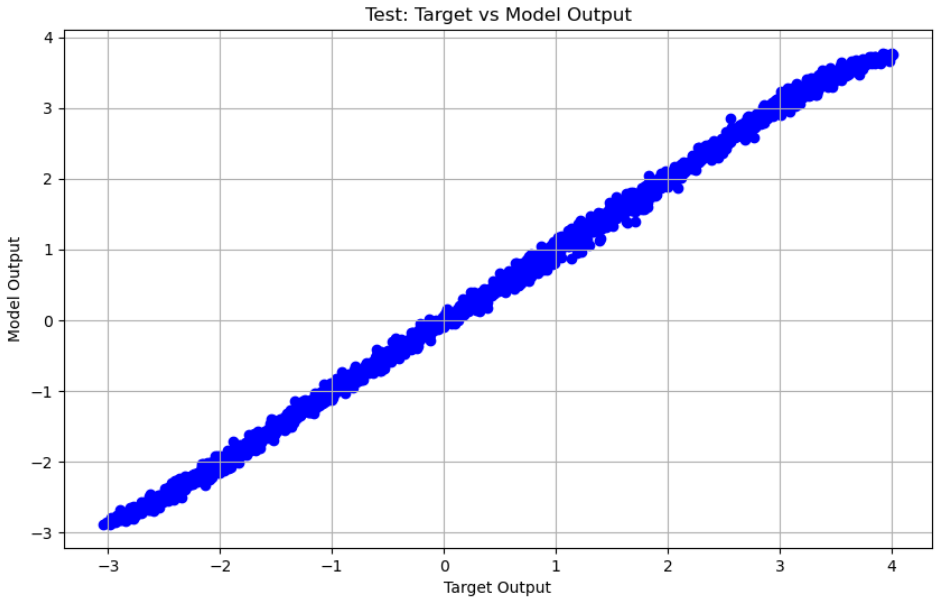


Fig. 30. Bivariate regression: target output versus model output on test data.

The bivariate scatter plots(i.e. Fig. 28, Fig. 29, Fig. 30) show a strong concentration of predictions around the target-output diagonal. The test RMSE of 0.080508 is close to the validation RMSE of 0.079254, which supports the conclusion that the selected model generalizes well.

C. Regression Comparison with Assignment I

The Assignment-I regression report showed substantial underfitting: the univariate single-neuron model produced an almost constant prediction near the target mean, while the bivariate model produced only a near-planar approximation to a curved surface. In Assignment II, the introduction of non-linear functions like hyperbolic tangent in the hidden layers changes the representational capacity. The selected 1-5-1 model learns the univariate hump with a test RMSE of

0.131105, and the selected 2-8-5-1 model learns the non-linear bivariate surface with a test RMSE of 0.080508. The improvement is therefore attributable to the non-linear hidden representation rather than simply increasing the training duration.

V. NODE-OUTPUT ANALYSIS

Node-output plots were generated for every hidden node and the final output node of the selected architecture for training, validation, and test data. For classification, the horizontal and vertical axes are the standardized input features X1 and X2 and the vertical axis is the node output. For univariate regression, X is plotted against the node output. For bivariate regression, X1 and X2 form the input plane and the node output forms the third axis. The neurons of initial layers learn the simple features and further layers combine the output of initial layers to learn global features as it can be observed in the plots.

These visualizations are retained in a separate supplementary report because the complete set is large. The supplied results archive contains 219 node-output figures: 174 for classification and 45 for regression. The main report therefore uses the decision-region and prediction plots to communicate the central findings, while the supplementary report provides the complete node-by-node evidence.

Supplementary Figure Report

VI. DISCUSSION AND INFERENCES

The experiments demonstrate three distinct effects. First, dataset geometry determines the minimum representation required for accurate learning. The linearly separable classification problem is solved by simple boundaries, whereas the non-linearly separable problem requires nonlinear feature transformations. Second, activation choice affects optimization. On the non-linearly separable data, tanh reaches a perfect solution at epoch 33 for the 2-10-8-3 architecture, while the best logistic model reaches 88.67% test accuracy after its best validation state at epoch 110. On the linearly separable data both activations reach 100% accuracy, but the tanh experiments reach their best states earlier than the logistic experiments.

Third, depth and hidden-node capacity are especially important for regression. In the univariate experiment, only the 1-5-1 architecture reaches a low-error solution, while the bivariate experiment shows a clear improvement when two hidden layers are introduced. The 2-8-5-1 architecture achieves a validation RMSE of 0.079254 and a test RMSE of 0.080508, compared with validation RMSE values above 0.40 for all one-hidden-layer bivariate alternatives.

The feature-space decision regions and regression surfaces also show why numerical metrics alone are insufficient. The classification decision regions reveal whether the model has learned the geometry of the classes, while the regression curves and surfaces reveal whether low error corresponds to learning the actual functional form. The node-output plots further show how individual hidden units transform the original inputs into non-linear intermediate representations.

One methodological point is important when interpreting the stopping epochs. Validation-based early stopping terminates a run when validation error increases, but the trainer retains and restores the weights associated with the lowest validation error. Thus, the reported model corresponds to the best observed validation state rather than the final update before termination.

VII. CONCLUSION

A reusable fully connected neural-network framework was implemented from scratch using dense layers, logistic and tanh activations, a linear regression output, squared-error loss, backpropagation, and pattern-mode stochastic gradient descent. The classification experiments confirm that the FCNN preserves perfect performance on the linearly separable dataset and substantially improves the non-linearly separable problem when sufficient non-linear depth is provided. The best non-linearly separable logistic model achieves 88.67% test accuracy, whereas the best non-linearly separable tanh model achieves 100%.

For regression, the selected 1-5-1 univariate model achieves test RMSE 0.131105 and the selected 2-8-5-1 bivariate model achieves test RMSE 0.080508. The target-versus-model plots and regression surfaces show that the FCNN learns the non-linear relationships that the Assignment-I single-neuron models could not represent. Overall, the experiments

establish that preprocessing, activation function, hidden-layer capacity, and validation-based model selection jointly determine the effectiveness of a from-scratch FCNN.

REFERENCES

[1] CS601T: Deep Learning, Programming Assignment II specification

[2] CS601T: Deep Learning course lecture material on fully connected neural networks, backpropagation, stochastic gradient descent, activation functions and parameter learning.

[3] Group 16, “Assignment 1 Report,” CS601T: Deep Learning